

S3 File Manager

A menu-driven tool for buckets and objects, built around the paginator pattern.

DURATION 2 h 15	SERVICES S3	DELIVERABLE s3_manager.py	PATTERN Paginat
---------------------------	-----------------------	-------------------------------------	---------------------------

1 Goal

Build an interactive command-line tool that manages S3 from Python. It creates and deletes buckets, uploads and downloads objects, lists them with a paginator, generates presigned URLs, and, as a challenge, backs up a whole local folder while preserving its structure.

TARGET MENU

```
=====
                        S3 MANAGER
=====
 1. Create Bucket      5. Download File
 2. List Buckets      6. Delete File
 3. Upload File       7. Generate Presigned URL
 4. List Files        8. Delete Bucket
 9. Backup Local Folder 0. Exit
=====

Choose an option: _
```

2 Learning objectives

- Explain why S3 has no folders and what a key prefix really is.
- Create a bucket correctly in `us-east-1` and in any other region.
- Choose between `upload_file` and `put_object`, and justify the choice.
- Use a paginator to list every object in a bucket, not just the first thousand.
- Generate a presigned URL and explain what it grants and for how long.
- Empty a bucket before deleting it.

3 Before you start

- [] Lab 1 runs successfully: your credentials work today.
- [] You re-copied your credentials for this session.
- [] You chose a bucket-name prefix nobody else will use (for example `youssef-lab2-`).
- [] You have a small local folder with a few files for the challenge.

Learner Lab rules

Create every bucket in `us-east-1`, which lets you skip the `CreateBucketConfiguration` branch (your code should still handle both cases). Keep Block Public Access on and share objects with presigned URLs. Prefix every bucket name with your own identifier, because bucket names are unique across all AWS accounts.

S3 has no folders

S3 is a flat key-value store. A key such as `backup/docs/notes.txt` is one single string, and the slash is only a convention. The Console and the `Prefix` filter simply group keys that share a prefix, which is what looks like a folder.

4 Step by step

Build incrementally

Start with the menu loop and stub functions that only print their own name. Then fill them in one at a time, testing each one from the menu.

Step 1: The menu loop

Write the loop and a dispatch dictionary before any AWS code. A dictionary mapping options to functions is cleaner than a long `if/elif` chain and easier to extend. Define the functions before building the dictionary.

```
import sys, boto3
from botocore.exceptions import ClientError

s3 = boto3.client("s3")
REGION = boto3.session.Session().region_name or "us-east-1"

ACTIONS = {"1": create_bucket, "2": list_buckets, ...}

def main():
    while True:
        print(MENU)
        choice = input("Choose an option: ").strip()
        if choice == "0":
            sys.exit(0)
        action = ACTIONS.get(choice)
        if action:
            action()
        else:
            print("Invalid option.")
```

Checkpoint

Every menu option prints its own name, and `0` exits cleanly.

Step 2: Create a bucket and handle the us-east-1 asymmetry

`us-east-1` is the original AWS region and the only one where passing a location constraint is an error. Every other region requires it. Write the branch once and move on.

Handle `BucketAlreadyExists` (someone else owns the name) separately from `BucketAlreadyOwnedByYou` (you already created it).

```
def create_bucket():
    name = input("New bucket name: ").strip()
    try:
        if REGION == "us-east-1":
            s3.create_bucket(Bucket=name)
        else:
            s3.create_bucket(
                Bucket=name,
                CreateBucketConfiguration={"LocationConstraint": REGION},
            )
        print(f"Created bucket '{name}'")
    except ClientError as e:
        code = e.response["Error"]["Code"]
        if code == "BucketAlreadyExists":
            print("That name is taken by another AWS account.")
        elif code == "BucketAlreadyOwnedByYou":
            print("You already own that bucket.")
        else:
            print(f"[AWS ERROR] {code}")
```

Checkpoint

Creating the same bucket twice gives a clear message the second time, not a traceback.

Step 3: Upload a file

Use `upload_file`, a managed transfer that splits large files into multipart uploads, sends the parts in parallel and retries failures.

Let the user choose the object key, defaulting to the file name. Check that the local file exists before calling AWS: a local mistake should not turn into an AWS error.

	upload_file	put_object
Nature	Managed transfer	Single API request
Large files	Multipart, parallel, automatic retries	You handle everything yourself (5 GB max per request)
Input	A file path or file object	Bytes or a file-like object
Best for	Files on disk	Small payloads built in memory

```
import os

def upload_file():
    path = input("Local file path: ").strip()
    bucket = input("Target bucket: ").strip()

    if not os.path.isfile(path):
        print(f"File not found: {path}")
        return

    default_key = os.path.basename(path)
    key = input(f"Object key [{default_key}]: ").strip() or default_key

    s3.upload_file(path, bucket, key)
    print(f"Uploaded {path} -> s3://{bucket}/{key}")
```

Checkpoint

The file appears in the bucket in the Console, under the key you chose.

Step 4: List objects with a paginator

This is the pattern of the day. `list_objects_v2` returns at most 1000 keys per response and gives a continuation token to fetch the rest. A paginator handles that bookkeeping for you.

Note `page.get("Contents", [])`: an empty bucket returns a page with no `Contents` key at all, so indexing it directly would crash.

```
def list_objects():
    bucket = input("Bucket name: ").strip()
    prefix = input("Prefix filter (Enter for all): ").strip()

    paginator = s3.get_paginator("list_objects_v2")
    pages = paginator.paginate(Bucket=bucket, Prefix=prefix)

    total_objects = 0
    total_bytes = 0

    for page in pages:
        for obj in page.get("Contents", []):
            total_objects += 1
            total_bytes += obj["Size"]
            print(f"{obj['Key']:<50} {obj['Size']:>12,}  "
                  f"{obj['LastModified']:%Y-%m-%d %H:%M}")

    print(f"\n{total_objects} object(s), "
          f"{total_bytes / 1024 / 1024:.2f} MB")
```

Checkpoint

Listing an empty bucket prints `(bucket is empty)` instead of raising `KeyError`. Listing with a prefix shows only the matching keys.

Step 5: Download and delete

`download_file` mirrors `upload_file`. For a missing object it raises a `ClientError` with code `404`: catch it and say so.

`delete_object` is idempotent. S3 reports success even for a key that never existed, by design.

```
def download_file():
    bucket = input("Bucket name: ").strip()
    key = input("Object key: ").strip()
    dest = input("Save as: ").strip() or os.path.basename(key)

    try:
        s3.download_file(bucket, key, dest)
        print(f"Downloaded -> {dest}")
    except ClientError as e:
        if e.response["Error"]["Code"] == "404":
            print("That object does not exist.")
        else:
            raise
```

DELETE A FILE

```
s3.delete_object(Bucket=bucket, Key=key)    # option 6, same style as the others
```

Checkpoint

You can upload a file, list it, download it under a new name, and open it.

Step 6: Generate a presigned URL

A presigned URL lets someone without an AWS account read a private object for a limited time. The signature is computed locally: the method makes no network call and will happily sign a URL for an object that does not exist.

Let the user choose the expiry, then paste the result in a browser and confirm the file downloads.

```
def generate_presigned_url():
    bucket = input("Bucket name: ").strip()
    key = input("Object key: ").strip()
    raw = input("Valid for how many seconds? [3600]: ").strip()
    expires = int(raw) if raw.isdigit() else 3600

    url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={"Bucket": bucket, "Key": key},
        ExpiresIn=expires,
    )
    print(f"\nValid for {expires} seconds:\n{url}\n")
```

Checkpoint

The URL opens the file in a browser. The plain <https://bucket.s3.amazonaws.com/key> address without a signature returns Access Denied: try both and see the difference.

Step 7: Delete a bucket

A bucket must be completely empty before it can be deleted. Handle `BucketNotEmpty` with a clear instruction, not an error dump.

Ask the user to type the bucket name again to confirm. A destructive action deserves a deliberate act, not a reflexive 'y'.

SUGGESTED SKELETON

```
def delete_bucket():
    name = input("Bucket to delete: ").strip()
    if input(f"Type '{name}' again to confirm: ").strip() != name:
        print("Names do not match. Aborted.")
        return
    try:
        s3.delete_bucket(Bucket=name)
        print(f"Deleted bucket '{name}'")
    except ClientError as e:
        if e.response["Error"]["Code"] == "BucketNotEmpty":
            print("The bucket is not empty. Delete its objects first.")
        else:
            raise
```

Checkpoint

Deleting a non-empty bucket tells you to empty it first. Deleting an empty one succeeds.

5 Expected output

Your formatting may differ, but the information must be the same.

```
Choose an option: 4
Bucket name: youssef-lab2-demo
Prefix filter (Enter for all):

KEY                                     SIZE  MODIFIED
-----
backup/docs/notes.txt                 1,204  2026-09-01 11:02
backup/images/logo.png               48,110  2026-09-01 11:02
report.csv                           8,940  2026-09-01 10:55

3 object(s), 0.06 MB total
```

6 Challenge: option 9, back up a local folder

Back up an entire local folder to S3 and preserve its structure as key prefixes.

1. Walk the folder recursively with `os.walk()`.
2. Convert each absolute path into a path relative to the chosen folder with `os.path.relpath()`.
3. Replace `os.sep` with a forward slash before using the result as a key.
4. Count successes and failures, and report both at the end.

```
for root, dirs, files in os.walk(folder):
    for filename in files:
        local_path = os.path.join(root, filename)

        relative = os.path.relpath(local_path, folder)
        key = prefix + relative.replace(os.sep, "/")

        s3.upload_file(local_path, bucket, key)
```

The slash replacement is not optional

On Windows `os.sep` is a backslash. A key containing backslashes becomes one object with a strange name instead of a folder structure.

7 Troubleshooting

You see	It means	Do this
<code>BucketAlreadyExists</code>	Another AWS account owns that name.	Bucket names are global. Add your own prefix.
<code>BucketAlreadyOwnedByYou</code>	You created it earlier.	Use it, or pick another name.
<code>InvalidBucketName</code>	Uppercase, underscore or wrong length.	Lowercase letters, digits, hyphens; 3 to 63 characters.
<code>IllegalLocationConstraint</code>	A constraint was sent for us-east-1.	Omit <code>CreateBucketConfiguration</code> in us-east-1.
<code>BucketNotEmpty</code>	Objects are still inside.	Delete every object, then the bucket.
<code>NoSuchBucket</code>	Wrong name, or wrong region.	Check both. Print <code>Session().region_name</code> .
<code>KeyError: 'Contents'</code>	The bucket or page is empty.	Use <code>page.get("Contents", [])</code> .

8 Cleanup

- [] Every bucket you created is emptied and deleted.
- [] The S3 Console shows no buckets belonging to you.
- [] Your script is committed to your Git repository.

9 Submission checklist

- [] `s3_manager.py` runs from a terminal with no editing.
- [] All eight menu options work, plus the backup challenge.
- [] Listing uses a paginator, not a bare `list_objects_v2` call.
- [] Bucket and object errors produce readable messages.
- [] The file is committed with a meaningful message.